

# Projet C++ : Dungeon Auto-Battle

## Simulation de combats de héros et tournois

Hyacinthe et Mohamed-Ali

L3 MIASHS - Programmation Orientée Objet

25 janvier 2026

- 1 Introduction et Objectifs
- 2 Architecture du Projet
- 3 Implémentation Technique
- 4 Scénario d'Exécution Détaillé
- 5 Les Bonus Ajoutés
- 6 Conclusion

# Introduction et Objectifs

## Le but du projet

Concevoir une simulation "Auto-Battler" où des équipes de héros s'affrontent de manière autonome, en respectant les principes SOLID et la gestion mémoire du C++.

## Compétences techniques mises en œuvre :

- **Programmation Orientée Objet (POO)** : Encapsulation stricte, Héritage.
- **Polymorphisme** : Utilisation de pointeurs génériques `Hero*` pour manipuler des classes dérivées.
- **Gestion de la mémoire** : Allocation dynamique (`new`) et libération rigoureuse (`delete`).
- **STL** : Utilisation intensive de `std::vector` pour la gestion d'équipe.

Pour garantir un code modulaire et professionnel, nous avons séparé la déclaration de l'implémentation :

- `include/` (Headers `.hpp`) : Contient les prototypes des classes et la structure d'héritage.
- `src/` (Sources `.cpp`) : Contient la logique métier et les algorithmes.
- `include/Utils.hpp` : Fichier utilitaire regroupant les inclusions standards (`iostream`, `vector`) pour alléger le code.

## Hiérarchie des Héros (6 Classes)

- **Hero** (Classe Abstraite) : Modèle de base.
  - → **Guerrier** : Mécanique de "Rage".
  - → **Mage** : Mécanique de "Perce-Armure".
  - → **Archer** : Mécanique de "Critique/Esquive".
  - → **Paladin** : Mécanique de "Tank" (Réduction fixe).
  - → **Assassin** : Mécanique "Glass Cannon" (Dégâts explosifs).
  - → **Druide** : Mécanique "Régénération" (Soin passif).

## Relations :

- **Agrégation** : Un Hero possède un pointeur vers un Objet.
- **Composition** : Un Joueur est responsable du cycle de vie de ses Hero.

# Polymorphisme et Abstraction

La méthode `calculerDegats` est virtuelle. Au moment de l'exécution, le programme choisit la bonne version (Guerrier ou Mage) même si le pointeur est de type `Hero*`.

```
1 class Hero {  
2     protected:  
3         int pv, attaque, defense;  
4     public:  
5         // Methode virtuelle pure : rend la classe abstraite  
6         virtual int getClasse() const = 0;  
7  
8         // Methode virtuelle redefinissable (Polymorphisme)  
9         virtual int calculerDegats(const Hero& cible) const;  
10  
11         virtual ~Hero(); // Destructeur virtuel essentiel !  
12 };
```

# Logique de Combat : Formule de Base

Avant toute spécialisation, tous les héros partagent une mécanique commune implémentée dans la classe mère Hero.

## Calcul Standard des Dégâts

La méthode de base `Hero::calculerDegats` applique la formule suivante :

$$\text{Dégâts} = \text{Attaque} - \left( \frac{\text{Défense Cible}}{2} \right)$$

## Règles importantes :

- **Atténuation** : On divise la défense par 2 pour éviter les combats interminables (0 dégât).
- **Minimum garanti** : Si le calcul donne  $\leq 0$ , les dégâts sont fixés à 1.

# Détail des Spécialisations : Guerrier et Archer

Nous avons implémenté des algorithmes distincts pour chaque classe :

## Guerrier : Logique Conditionnelle

**Concept** : La Rage du Berserker.

- Utilise une condition d'état :  
if (pv < 30%).
- **Effet** : Multiplie les dégâts par 1.2 si la condition est remplie.

## Archer : Logique Probabiliste

**Concept** : Le Coup Critique.

- Utilise le hasard à chaque tir :  
rand() % 100 < 20.
- **Effet** : 20% de chance de doubler les dégâts finaux.



# Détail des Spécialisations : Mage et Paladin

## Mage : Logique Arithmétique

**Concept** : Magie Instable  
Perce-Armure.

- **Calcul** : Ignore 30% de la défense adverse.
- **Variation** : Ajoute un facteur aléatoire ( $\pm 20\%$ ) au résultat.

## Paladin : Logique Défensive

**Concept** : Le Tank (Override).

- Ne modifie pas l'attaque, mais surcharge perdrePV().
- **Effet** : Réduit systématiquement les dégâts reçus d'une valeur fixe.

# Détail des Spécialisations : Assassin et Druide

## Assassin : Glass Cannon

**Concept** : Risque élevé /  
Récompense élevée.

- **Stats** : Beaucoup de vitesse/attaque, très peu de vie.
- **Backstab** : Chance d'infliger +50% de dégâts en ignorant une partie de l'armure.

## Druide : Régénération

**Concept** : Survivabilité Naturelle.

- Surcharge de perdrePV().
- **Effet** : Après avoir reçu un coup, il récupère automatiquement +5 PV (Peau d'écorce).

La classe Joueur agit comme le "Garbage Collector" de son équipe.

```
1 // Destructeur de Joueur
2 Joueur::~~Joueur() {
3     cout << "Destruction de l'equipe de " << nom << endl;
4
5     // On parcourt le vecteur de pointeurs
6     for (size_t i = 0; i < equipe.size(); ++i) {
7         // Indispensable car les heros ont ete crees avec '
8         delete equipe[i];
9     }
10    equipe.clear();
11 }
```

# 1. Initialisation : La Factory en action

Au lancement du programme, la fonction `lancerTournoi()` initialise 4 joueurs.

## Génération d'un Héros (HeroFactory)

- 1 Tirage aléatoire du type parmi les **6 classes** disponibles.
- 2 Allocation dynamique via `new`.
- 3 **Bonus Objet** : Une chance sur trois de générer un `Objet` (ex : Épée de Feu) et de l'équiper immédiatement via le pointeur `equipement`.
- 4 Retourne un `Hero*` au Joueur.

## 2. La Boucle de Combat (Le Moteur)

Lors d'un duel entre  $h1$  et  $h2$ , voici l'algorithme exécuté à chaque tour :

- ❶ **Test d'Initiative** : Calcul basé sur la vitesse  $\times$  rand(). Le plus rapide attaque en premier.
- ❷ **Attaque** : L'attaquant appelle `calculerDegats()`.
  - *Logique* : Applique la formule de base ou la spécialisation (Polymorphisme).
- ❸ **Défense** : Le défenseur appelle `perdrePV()`.
  - *Polymorphisme* : Le Paladin réduit les dégâts, le Druide se régénère.
- ❹ **Vérification** : Si  $PV \leq 0$ , le combat s'arrête.

### 3. Logique de Tournoi et Soins

Le système gère un arbre de tournoi complet :

**Demi-finales → Finale**

#### Règle de Gestion des Soins

Entre chaque combat, une étape critique survient :

- L'équipe perdante est éliminée.
- L'équipe gagnante appelle la méthode `soigner()` sur tous ses héros survivants pour remettre leurs PV au max avant le prochain tour.

## Gameplay

- **Classes Avancées** : Ajout de l'Assassin et du Druide (6 classes au total).
- **Objets** : Système d'équipement dynamique qui modifie les stats (Attaque, Défense, Vitesse).

## Expérience Utilisateur

- **Mode Tournoi** : Simulation complète à 4 joueurs.
- **Affichage** : Codes couleurs ANSI (Rouge=Dégâts, Vert=Vie) pour une lecture fluide des logs.

## Bilan du projet :

- ✓ Une architecture robuste et extensible.
- ✓ Une gestion de la mémoire sans fuite (vérifiée par les destructeurs).
- ✓ Ajout facile de nouvelles classes (Assassin/Druide) sans casser le moteur.

**Merci de votre attention. Place à la démonstration !**